CarSharing Dalmine

Iterazione finale

Sistema intelligente di Car Sharing ottimizzato per il polo universitario di Dalmine

Sommario

1. Scopo dell'Iterazione 1
2. Analisi dei Requisiti 2
  2.1 Requisiti Funzionali 2
    RF01 — Caricamento configurazione applicativa 2
    RF02 — Selezione modalità di esecuzione da CLI 3
    RF03 — Modalità POPOLA (download dati e popolamento cache) 4
    RF04 — Modalità OTTIMIZZA (calcolo equipaggi e generazione mappa) 5
  2.2 Requisiti Non Funzionali 6
3. Design dell'Architettura — Modello UML 4+1 Viste 7
  3.1 Vista Logica (Class Diagram) 7
  3.2 Vista Fisica (Deployment / Component View) 7
  3.3 Vista di Sviluppo (Development / Package View) 8
4. Design in Piccolo — Pseudocodice e Complessità 9
  4.1 — execute_optimize_mode(...) 9
  4.2 — MapGenerator.generate(...) 10
  4.3 — ConsoleFormatter.format_flotta(...) 11
4. Analisi Dinamica — Test e Copertura 13
  4.1 Obiettivi e Scope 13
  4.2 Strategia di Test 13
  4.3 Esempi di Test Rappresentativi 14
    Esempio 1: Unit — AppConfig.from_env_and_file() senza API key 14
    Esempio 2: Unit — Routing di main() in modalità POPOLA 14
    Esempio 3: Unit — Parsing CLI con modalità ottimizza 15
    Esempio 4: Unit — Generazione mappa con fallback coordinate 15
    Esempio 5: Unit — Formattazione di una flotta non vuota 15
  4.4 Risultati Copertura 16
  4.5 Riepilogo Test 16
5. Analisi Statica — Metriche 17
  5.1 Strumenti e Target 17
  5.2 Maintainability Index 17
  5.3 Complessità Ciclomatica (radon cc src/ -s -a) 18
  5.4 Qualità Codice (pylint src/) 20
  5.5 Riepilogo Metriche 20
6. Installazione ed Esecuzione 21
  6.1 Prerequisiti 21
  6.2 Installazione dipendenze 21

6.3 Configurazione variabili d'ambiente 21
6.4 Esecuzione del programma 21

1. Scopo dell'Iterazione

L'Iterazione 4 introduce e consolida:

- **Configurazione applicativa centralizzata (*src/config.py*)**: gestione unificata dei percorsi su filesystem (*PathConfig*) e dei parametri business/tecnici (*AppConfig*) con caricamento da variabili d'ambiente e file JSON (*data/config/app_config.json*).
- **Entry point applicativo (*src/main.py*)**: orchestrazione dei 4 layer architetturali (*Data Access, Services, Business, Presentation)*.
- **Presentation Layer** (src/presentation/):
  - parsing degli argomenti CLI (CLIArgumentParser),
  - formattazione testuale del risultato (ConsoleFormatter),
  - generazione della mappa HTML interattiva con Folium (MapGenerator).
- **Test di integrazione a livello applicativo e di presentazione** (tests/test_application/, tests/test_presentation/): verificano il corretto wiring tra componenti e il rispetto dei contratti esterni (CLI, formattazione, generazione mappa).

Questa iterazione consolida il **contratto architetturale** tra:

- *AppConfig* / *PathConfig* e tutti i layer che dipendono da path e parametri business;
- main.py e gli orchestratori di business (*PopulateOrchestrator*, *OptimizeOrchestrator*);
- il **Presentation Layer** e il resto del sistema, tramite:
  - interfaccia CLI,
  - oggetti di dominio (*Equipaggio*, *Studente*) per formattazione e visualizzazione,
  - repository cache per la costruzione della mappa HTML.

2. Analisi dei Requisiti

2.1 Requisiti Funzionali

RF01 — Caricamento configurazione applicativa

Priorità: Alta

Descrizione:
Il sistema deve caricare all'avvio la configurazione applicativa, combinando variabili d'ambiente e file JSON, e mettere a disposizione dei layer inferiori:

- i parametri di business (capacità auto, numero cluster, max deviazione, bonus corso di laurea, comune di destinazione);
- i parametri tecnici (batch size chiamate API, jitter geografico, livelli di logging, ambiente);
- la chiave API ORS_API_KEY in modo sicuro (non serializzata nei log o nei report).

Formato input:

- **Variabili d'ambiente**:

```
ORS_API_KEY=<stringa_non_vuota>
```

- **File JSON** (data/config/app_config.json):

```
{
  "nominatim_user_agent": "carsharing-dalmine-unibg-v1",
  "capacita_macchina": 4,
  "bonus_corso_laurea": 180,
  "max_deviazione_sec": 900,
  "numero_cluster": 7,
  "comune_destinazione": "DALMINE",
  "batch_size_chiamate": 100,
  "jitter_factor": 0.0003,
  "log_level": "INFO",
  "environment": "development"
}
```

Passi:

1. All'avvio, AppConfig.from_env_and_file() viene invocato da main().
2. Il metodo tenta di leggere il file JSON app_config.json se esiste.
3. Il metodo legge la variabile d'ambiente ORS_API_KEY.
4. Se ORS_API_KEY non è definita, viene sollevato un ValueError e il programma termina con codice 1.
5. Se la variabile è presente, il metodo costruisce un'istanza AppConfig combinando:
    - i valori di default definiti nella dataclass,
    - i valori presenti nel file JSON (se esistente),
    - la chiave ORS_API_KEY letta dall'ambiente.
6. main() stampa un riepilogo della configurazione (senza mostrare la chiave) e prosegue con l'inizializzazione dei layer.

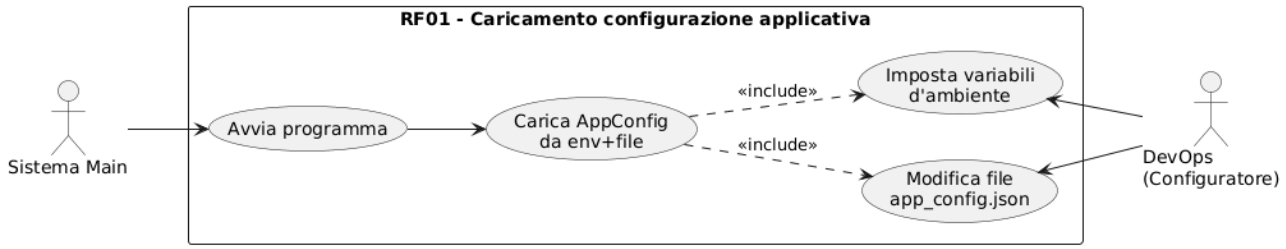
Precondizioni:

- La variabile d'ambiente ORS_API_KEY deve essere impostata.
- Il file data/config/app_config.json, se presente, deve contenere JSON valido (in caso contrario è loggato un warning ma non interrompe l'esecuzione).

Postcondizioni:

- È disponibile un oggetto AppConfig coerente, con chiave API valida e parametri business/tecnici consistenti.

Diagramma Use Case:



RF02 — Selezione modalità di esecuzione da CLI

Priorità: Alta

Descrizione:
L'utente deve poter scegliere, da riga di comando, la modalità di esecuzione del sistema (popola oppure ottimizza), in modo che main() possa effettuare il routing verso l'orchestratore corrispondente.

Formato input:

```
python src/main.py popola

python src/main.py ottimizza
```

Passi:

1. L'utente esegue il programma passando un argomento posizionale modalita.
2. CLIArgumentParser.parse() legge sys.argv e valida che modalita ∈ {popola, ottimizza}.
3. In caso di valore valido, viene restituita una ProgramConfig(mode=...).
4. main() controlla program_config.mode:
    - se "popola", invoca execute_populate_mode(...);
    - se "ottimizza", invoca execute_optimize_mode(...);
    - altrimenti, stampa un errore e termina con sys.exit(1).

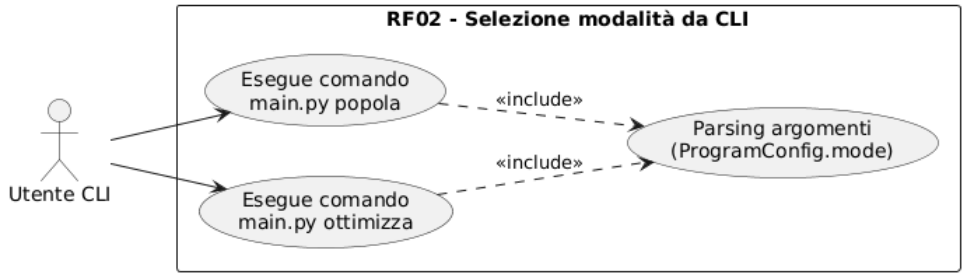
Precondizioni:

- L'utente deve passare un argomento posizionale alla CLI.

Postcondizioni:

- Il sistema ha selezionato una delle due modalità valide, oppure ha terminato con errore in caso di valore non ammesso.

Diagramma Use Case:



RF03 — Modalità POPOLA (download dati e popolamento cache)

Priorità: Alta

Descrizione:
In modalità popola il sistema deve:

- geocodificare le località degli studenti;
- eseguire clustering geografico;
- contattare le API di routing per scaricare i percorsi;
- salvare i dati nelle cache su filesystem.

Passi principali (dal punto di vista del Presentation/Application Layer):

1. main() inizializza i layer e costruisce populate_orch (via initialize_layer3_business()).
2. main() chiama execute_populate_mode(populate_orchestrator, app_config).
3. La funzione stampa un riepilogo della modalità (POPOLA) e i passi che verranno eseguiti.
4. populate_orchestrator.execute(batch_size=app_config.batch_size_chiamate) viene invocato.
5. In caso di successo, viene stampato un messaggio di completamento e un suggerimento per la modalità ottimizza.
6. In caso di KeyboardInterrupt, il sistema informa che i dati già scaricati sono stati salvati in cache.
7. In caso di eccezioni generiche, viene stampato lo stack trace e il programma termina con codice di errore.

Precondizioni:

- Connessione Internet attiva.
- ORS_API_KEY valida e configurata.
- File di input studenti disponibile.

Postcondizioni:

- Le cache di coordinate e percorsi sono state popolate (parzialmente o completamente, a seconda della riuscita).

RF04 — Modalità OTTIMIZZA (calcolo equipaggi e generazione mappa)

Priorità: Alta

Descrizione:
In modalità ottimizza il sistema deve:

- caricare i dati dalla cache;
- eseguire clustering e ottimizzazione greedy per costruire gli equipaggi;
- stampare sul terminale una rappresentazione leggibile della flotta;
- generare una mappa HTML con percorsi e marker degli studenti.

Passi principali:

1. main() inizializza i layer e costruisce optimize_orch, console_formatter, map_generator, cache_repo.
2. main() chiama execute_optimize_mode(optimize_orch, console_formatter, map_generator, cache_repo).
3. La funzione stampa una descrizione della modalità OTTIMIZZA.
4. Viene avviato un timer (time.perf_counter()).
5. flotta = optimize_orchestrator.execute() calcola gli equipaggi a partire dalla cache.
6. Viene calcolato il tempo di esecuzione.
7. console_formatter.print_flotta(flotta) stampa gli equipaggi in modo ordinato.
8. Vengono calcolate statistiche aggregate (numero studenti, numero auto, occupazione media).
9. map_generator.generate(flotta, PathConfig.mappa_html_file(), cache_repo) genera la mappa HTML.
10. Viene stampato il percorso del file HTML generato.

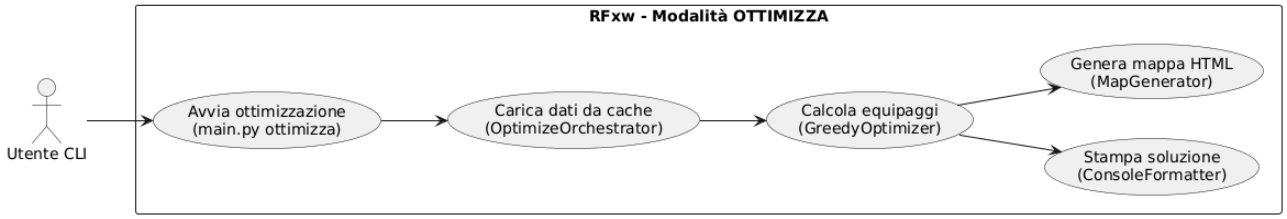
Precondizioni:

- La cache deve essere valida e completa per la destinazione configurata; in caso contrario viene sollevato ValueError.

Postcondizioni:

- È stata prodotta una lista di equipaggi coerente con i vincoli di business.
- È stato generato un file HTML visualizzabile in un browser con percorsi e marker.

Diagramma Use Case:



2.2 Requisiti Non Funzionali

RNF01 — Sicurezza della configurazione

- **Metrica**: la chiave ORS_API_KEY non deve mai comparire in chiaro in log, output a video o report esportati.
- **Motivazione**: separare le credenziali dal codice sorgente e dai log per evitare leak accidentali.

Implementazione rilevante:

- AppConfig.to_dict() sostituisce il valore reale di ors_api_key con la stringa "***HIDDEN***".

RNF02 — Manutenibilità della struttura dati

- **Metrica**: tutti i path verso dati di configurazione, input, cache e output devono essere centralizzati in PathConfig; nessun path hard-coded all'interno dei layer applicativi.
- **Motivazione**: facilitare il refactoring della struttura delle directory (es. cambio di radice data/) senza modificare il codice dei servizi e della business logic.

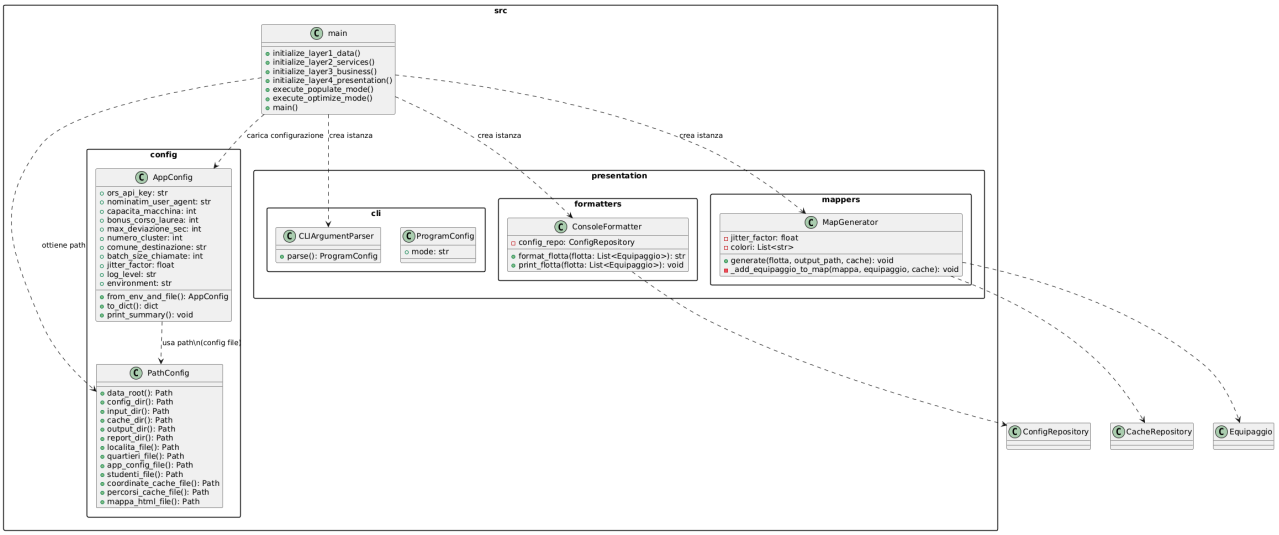
RNF03 — Usabilità CLI

- **Metrica**: il comando deve offrire help testuale chiaro (python src/main.py -h) e validare i valori della modalità con messaggi d'errore espliciti.
- **Motivazione**: ridurre gli errori di utilizzo e rendere autoesplicative le due modalità popola e ottimizza.

3. Design dell'Architettura — Modello UML 4+1 Viste

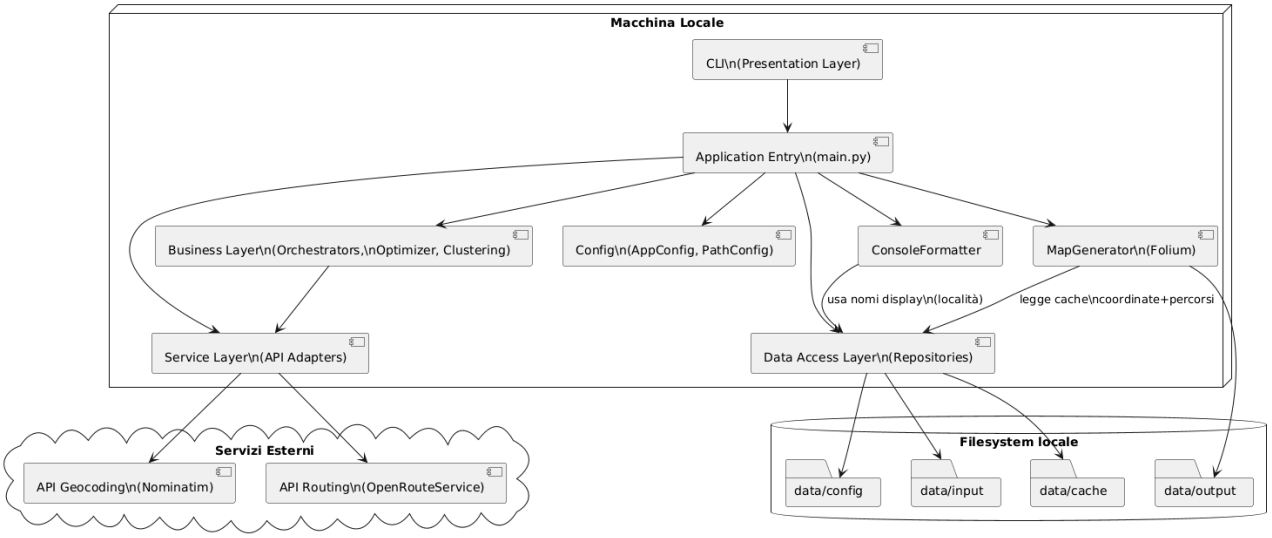
3.1 Vista Logica (Class Diagram)

Di seguito un estratto delle classi più rilevanti introdotte/estese in questa iterazione.

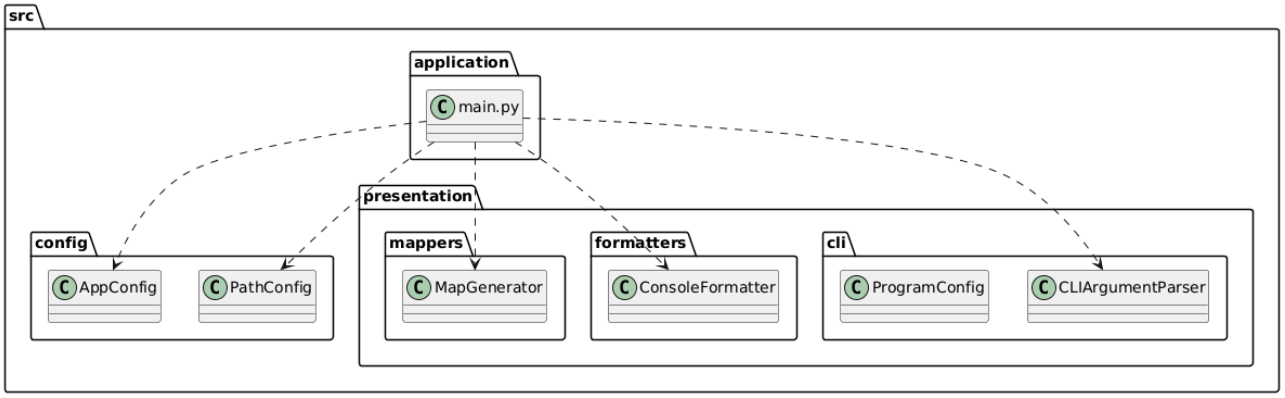


3.2 Vista Fisica (Deployment / Component View)

Questa iterazione non modifica sostanzialmente il deployment rispetto alle iterazioni precedenti, ma dettaglia i componenti del Presentation Layer e della configurazione.



3.3 Vista di Sviluppo (Development / Package View)



4. Design in Piccolo — Pseudocodice e Complessità

In questa iterazione risultano particolarmente significative, dal punto di vista della logica, le funzioni:

- execute_optimize_mode(...)
- MapGenerator.generate(...)
- ConsoleFormatter.format_flotta(...)

4.1 — execute_optimize_mode(...)

Componente: *execute_optimize_mode*

```
FUNZIONE execute_optimize_mode(optimize_orchestrator, console_formatter, map_generator, cache_repo):
  INPUT:
    - optimize_orchestrator: coordinatore della logica di business per l'ottimizzazione
    - console_formatter: formatter testuale della flotta
    - map_generator: generatore di mappe HTML
    - cache_repo: repository cache, usato dal map_generator
  OUTPUT: nessuno (effetti collaterali: stampa e generazione file)

  stampa intestazione "MODALITÀ: OTTIMIZZA" e descrizione passi

  PROVA:
    start_time ← time.perf_counter()
    flotta ← optimize_orchestrator.execute()
    end_time ← time.perf_counter()
    elapsed ← end_time - start_time

    stampa separatore
    console_formatter.print_flotta(flotta)

    totale_studenti ← somma di equipaggio.capacita_utilizzata per ogni equipaggio in flotta
    SE flotta non è vuota ALLORA
      avg_occupazione ← totale_studenti / len(flotta)
    ALTRIMENTI
      avg_occupazione ← 0
    FINE SE

    stampa statistiche:
      - totale_studenti
      - len(flotta)
      - avg_occupazione
      - elapsed

    stampa messaggio "Generazione mappa HTML..."
    output_path ← PathConfig.mappa_html_file()
    map_generator.generate(flotta, output_path, cache_repo)

    stampa messaggi di completamento + path del file HTML

  CATTURA ValueError come e:
    stampa "ERRORE DI VALIDAZIONE" + messaggio di e
    stampa "La cache è incompleta o mancante"
    termina processo con codice 1

  CATTURA Exception generica come e:
    stampa "ERRORE durante ottimizzazione" + e
    stampa stack trace
    termina processo con codice 1
FINE FUNZIONE
```

Analisi Complessità:

- **Tempo**: dominato da *optimize_orchestrator.execute()* (algoritmi di clustering e ottimizzazione) e da *map_generator.generate()*. Dal punto di vista della funzione di orchestrazione:
  - calcolo di statistiche e stampa è $O(|flotta|)$
- **Spazio**: $O(|flotta|)$, determinato dalla struttura dati degli equipaggi, già costruita dall'orchestrator.
- **I/O**:
  - stampa su console proporzionale a $(|flotta| + |numero_studenti|)$
  - 1 scrittura del file HTML della mappa più accessi di lettura alla cache (interni a MapGenerator).

4.2 — MapGenerator.generate(...)

Componente: MapGenerator.generate(flotta, output_path, cache)

```
FUNZIONE generate(flotta, output_path, cache):
  INPUT:
    - flotta: lista di equipaggi da rappresentare
    - output_path: percorso file HTML da generare
    - cache: CacheRepository, con coordinate e percorsi
  OUTPUT: nessuno (scrive file HTML)

  coords_dalmine ← cache.coordinate_cache.get("DALMINE")
  SE coords_dalmine è None ALLORA
    coords_dalmine ← (45.695, 9.67)  // fallback
  FINE SE

  mappa ← folium.Map(location=coords_dalmine, zoom_start=11)

  aggiungi Marker destinazione (Dalmine) alla mappa

  PER ogni equipaggio IN flotta:
    chiama _add_equipaggio_to_map(mappa, equipaggio, cache)
  FINE PER

  crea output_path.parent se non esiste (mkdir -p)
  salva la mappa in output_path
  stampa "Mappa generata: output_path"
FINE FUNZIONE
La funzione ausiliaria _add_equipaggio_to_map:
FUNZIONE _add_equipaggio_to_map(mappa, equipaggio, cache):
  colore ← colori[equipaggio.id modulo len(colori)]

  // Disegno dei percorsi
  PER i da 0 a len(equipaggio.percorso_tappe) - 2:
    partenza ← equipaggio.percorso_tappe[i]
    arrivo ← equipaggio.percorso_tappe[i+1]
    percorso ← cache.percorsi_cache.get(partenza, arrivo)
    SE percorso esiste E percorso.geometria non è vuota ALLORA
      coords_invertite ← lista di (lat, lon) per ogni (lon, lat) in percorso.geometria
      aggiungi PolyLine a mappa con coords_invertite e colore
    FINE SE
  FINE PER

  // Marker dei membri
  PER ogni membro IN equipaggio.membri:
    coords ← cache.coordinate_cache.get(membro.localita)
    SE coords esiste ALLORA
      lat_jitter ← coords[0] + random.uniform(-jitter_factor, jitter_factor)
      lon_jitter ← coords[1] + random.uniform(-jitter_factor, jitter_factor)
      aggiungi Marker alla mappa con posizione (lat_jitter, lon_jitter),
```

```
        popup con email e località, tooltip con info equipaggio
    FINE SE
  FINE PER
FINE FUNZIONE
```

Analisi Complessità:

- **Tempo**:
  - per ogni equipaggio (e), *_add_equipaggio_to_map* visita tutte le tappe e tutti i membri;
  - complessivamente $O\left(\sum_e \left(\left|percorso_tappe_e\right| + \left|membri_e\right|\right)\right)$.
- **Spazio**:
  - la mappa Folium mantiene in memoria tutte le geometrie aggiunte $O(|segmenti| + |marker|)$.
- **I/O**:
  - lettura da *coordinate_cache* e *percorsi_cache* per ogni tappa/membro,
  - una scrittura del file HTML di output.

4.3 — ConsoleFormatter.format_flotta(...)

Componente: ConsoleFormatter.format_flotta(flotta)

```
FUNZIONE format_flotta(flotta: List[Equipaggio]) -> stringa:
  INPUT: lista di equipaggi
  OUTPUT: stringa formattata da stampare a video

  SE flotta è vuota ALLORA
    RITORNA "Nessuna auto creata."
  FINE SE

  lines ← lista con elemento iniziale "=== SOLUZIONE FINALE ==="

  PER ogni equipaggio IN flotta:
    membri_ordinati ← lista di equipaggio.membri
    ordina membri_ordinati per localita in ordine decrescente

    header ← stringa "--- Auto {id} ({capacita_utilizzata} persone) ---"
    aggiungi header a lines

    PER ogni membro IN membri_ordinati:
      loc_display ← config_repo.get_localita_display_name(membro.localita)
      riga ← "  - {email} | Da: {loc_display} | Corso: {corso}"
      aggiungi riga a lines
    FINE PER

    aggiungi riga di separazione "-" * len(header) a lines
    aggiungi riga vuota a lines
  FINE PER

  RITORNA join(lines, "\\n")
FINE FUNZIONE
```

Analisi Complessità:

- **Tempo**:
  - per ogni equipaggio, ordinamento dei membri: $O\left(\left|membri_e\right| \log log \left|membri_e\right|\right)$

- complessivamente $O\left(\sum_e \left(\left|percorso_tappe_e\right| + \left|membri_e\right|\right)\right)$.

- **Spazio**:
  - $O\left(\sum_e \left|membri_e\right|\right)$ per la costruzione delle stringhe di output.
- **I/O**:
  - nessun I/O esterno; solo ritorno di una stringa, che viene successivamente scritta su stdout da *print_flotta*.

5. Analisi Dinamica — Test e Copertura

5.1 Obiettivi e Scope

In questa iterazione i test si concentrano su:

- **Configurazione**:
  - corretto wiring di PathConfig e dei path dei file;
  - comportamento di AppConfig.from_env_and_file() in assenza/presenza di variabili d'ambiente e file JSON.
- **Application Layer (main.py)**:
  - corretto wiring dei 4 layer;
  - routing sulla base di ProgramConfig.mode (popola, ottimizza, modalità sconosciuta);
  - gestione di errori di configurazione.
- **Presentation Layer**:
  - parsing degli argomenti CLI (CLIArgumentParser e ProgramConfig);
  - generazione delle mappe (MapGenerator) senza toccare Folium reale (uso intensivo di mocking);
  - formattazione e stampa della flotta (ConsoleFormatter).

Target generali:

- **Statement coverage ≥ 85%** (sull'intero src/).
- **Branch coverage ≥ 80%** (per i branch principali di configurazione e routing).

5.2 Strategia di Test

La strategia adottata segue lo schema:

Tipo	Scope	Caratteristiche	Marker
Unit	config, presentation, funzioni di wiring in main	Nessun accesso a rete o filesystem reale; uso di monkeypatch e oggetti dummy	@pytest.mark.unit (marker logici, se usati)
Integration (light)	main.main() con orchestratori/wiring fittizi	Testano il flusso end-to-end dell'entry point, sostituendo la logica pesante con stub/dummy	(eventuale marker @pytest.mark.integration)

Strumenti:

- pytest per il framework di test.
- pytest.monkeypatch per sostituire funzioni, classi e variabili d'ambiente.
- unittest.mock.MagicMock e patch per spiare interazioni con Folium e repository cache.
- fixture tmp_path di pytest per lavorare su filesystem temporaneo.

- fixture capsys per catturare l'output su stdout.

Regola mocking applicata:

- le dipendenze vengono patchate **nel modulo che le usa** (es. src.presentation.mappers.map_generator.folium, main_mod.ConfigRepository, main_mod.AppConfig), non nel modulo dove sono definite.

5.3 Esempi di Test Rappresentativi

Esempio 1: Unit — AppConfig.from_env_and_file() senza API key

Pattern: AAA + monkeypatch su variabili d'ambiente

```
def test_from_env_and_file_raises_without_api_key(self, monkeypatch, tmp_path):
    # Arrange: puntiamo il file di config a una path inesistente
    config_file = tmp_path / "config" / "app_config.json"
    monkeypatch.setattr("src.config.PathConfig.app_config_file", lambda: config_file)
    monkeypatch.delenv("ORS_API_KEY", raising=False)

    # Act + Assert
    with pytest.raises(ValueError):
        AppConfig.from_env_and_file()
```

Copertura:

Copre il ramo in cui ORS_API_KEY non è definita e viene sollevato ValueError in from_env_and_file().

Esempio 2: Unit — Routing di main() in modalità POPOLA

Pattern: AAA + Dummy Object + monkeypatch

```
def test_main_routing_modalita_popola(monkeypatch):
    _setup_minimal_main_environment(monkeypatch, mode="popola")

    called = {"popola": False, "ottimizza": False}

    def fake_execute_popola(orchestrator, app_config):
        called["popola"] = True

    def fake_execute_ottimizza(opt_orch, console_formatter, map_gen, cache_repo):
        called["ottimizza"] = True

    monkeypatch.setattr(main_mod, "execute_populate_mode", fake_execute_popola)
    monkeypatch.setattr(main_mod, "execute_optimize_mode", fake_execute_ottimizza)

    main_mod.main()

    assert called["popola"] is True
    assert called["ottimizza"] is False
```

Copertura:

Copre il ramo *if program_config.mode == "popola"* in *main(),* assicurando che venga invocata soltanto *execute_populate_mode*.

Esempio 3: Unit — Parsing CLI con modalità ottimizza

Pattern: AAA + monkeypatch di sys.argv

```
def test_parse_modalita_ottimizza(self, monkeypatch):
    monkeypatch.setattr("sys.argv", ["script", "ottimizza"])
    parser = CLIArgumentParser()

    result = parser.parse()

    assert isinstance(result, ProgramConfig)
    assert result.mode == "ottimizza"
```

Copertura:

Copre il percorso di parsing valido in *CLIArgumentParser.parse()* per la modalità ottimizza.

Esempio 4: Unit — Generazione mappa con fallback coordinate

Pattern: AAA + MagicMock + patch di folium

```
def test_generate_fallback_coords_se_dalmine_mancante(self, tmp_path, monkeypatch):
    output_path = tmp_path / "mappa.html"
    mock_map = MagicMock()
    mock_folium = MagicMock()
    mock_folium.Map.return_value = mock_map

    coord_cache = MagicMock()
    coord_cache.get.return_value = None  # DALMINE non in cache

    cache_repo = MagicMock()
    cache_repo.coordinate_cache = coord_cache
    cache_repo.percorsi_cache = MagicMock()
    cache_repo.percorsi_cache.get.return_value = None

    gen = MapGenerator()

    with patch("src.presentation.mappers.map_generator.folium", mock_folium):
        gen.generate([], output_path, cache_repo)

    call_args = mock_folium.Map.call_args
    assert call_args.kwargs.get("location") == (45.695, 9.67)
```

Copertura:

Copre il ramo di fallback in *MapGenerator.generate()* quando le coordinate di “DALMINE” non sono presenti in cache.

Esempio 5: Unit — Formattazione di una flotta non vuota

Pattern: AAA

```
def test_format_flotta_una_auto_un_membro(self):
    mapping = {"BERGAMO": "Bergamo", "INGEGNERIA": "Ingegneria"}
    repo = DummyConfigRepo(mapping)
    formatter = ConsoleFormatter(config_repo=repo)
```

```
    autista = _make_studente("a@unibg.it", "BERGAMO", "INGEGNERIA")
    eq = _make_equipaggio(1, autista, tappe=["BERGAMO", "DALMINE"])

    result = formatter.format_flotta([eq])

    assert "=== SOLUZIONE FINALE ===" in result
    assert "Auto 1 (1 persone)" in result
    assert "a@unibg.it" in result
    assert "Bergamo" in result
    assert "INGEGNERIA" in result
```

Copertura:

Copre il percorso principale di *ConsoleFormatter.format_flotta()* per una flotta con un solo equipaggio e un solo membro.

5.4 Risultati Copertura

Modulo	Stmts	Miss	Branch	BrPart	Cover
src/config.py	91	12	4	0	87%
src/main.py	145	76	6	1	49%
src/presentation/cli/argument_parser.py	10	0	0	0	100%
src/presentation/mappers/map_generator.py	36	2	12	2	92%
src/presentation/formatters/console_formatter.py	22	0	6	0	100%
TOTALE	**798**	**98**	**124**	**6**	**89%**

Nota sulla copertura di *main.py*: il modulo presenta una copertura di statement del 49%, inferiore al target generale. Questo è atteso: la maggior parte delle righe non coperte appartengono alle funzioni initialize_layer1..4(), che istanziano componenti concreti con dipendenze esterne (filesystem, API). Queste righe vengono deliberatamente escluse dai test unitari per evitare effetti collaterali, e vengono invece verificate indirettamente tramite i test di integrazione che usano stub.

La copertura complessiva del progetto rimane all'89%, al di sopra del target ≥85%.

5.5 Riepilogo Test

Aspetto	Valore
Totale test	164 (163 passed, 1 skipped)
Statement coverage target	≥ 85%
Branch coverage target	≥ 80%
Tempo esecuzione	~5.5 s

6. Analisi Statica — Metriche

6.1 Strumenti e Target

Metrica	Target	Strumento
Cyclomatic Complexity	≤ 10 per funzione (grado ≤ B)	radon cc
Maintainability Index	≥ 60 per modulo (grado A)	radon mi
Pylint Score	≥ 8.5/10	pylint

6.2 Maintainability Index

Valori ricavati dall'esecuzione di *radon mi src/ -s*.

```
src\config.py - A (62.74)
src\main.py - A (61.08)
src\__init__.py - A (100.00)
src\business\__init__.py - A (100.00)
src\business\clustering\base.py - A (100.00)
src\business\clustering\kmeans_clustering.py - A (90.96)
src\business\clustering\__init__.py - A (100.00)
src\business\optimization\base.py - A (100.00)
src\business\optimization\greedy_optimizer.py - A (64.81)
src\business\optimization\__init__.py - A (100.00)
src\business\orchestrators\optimization_facade.py - A (100.00)
src\business\orchestrators\optimize_orchestrator.py - A (83.11)
src\business\orchestrators\populate_orchestrator.py - A (70.51)
src\business\orchestrators\__init__.py - A (100.00)
src\business\validators\data_validator.py - A (82.93)
src\business\validators\__init__.py - A (100.00)
src\data\__init__.py - A (100.00)
src\data\models\equipaggio.py - A (87.95)
src\data\models\percorso.py - A (87.43)
src\data\models\studente.py - A (100.00)
src\data\models\__init__.py - A (100.00)
src\data\repositories\cache_repository.py - A (66.18)
src\data\repositories\config_repository.py - A (73.66)
src\data\repositories\studenti_repository.py - A (85.07)
src\data\repositories\__init__.py - A (100.00)
src\presentation\__init__.py - A (100.00)
src\presentation\cli\argument_parser.py - A (100.00)
src\presentation\cli\__init__.py - A (100.00)
src\presentation\formatters\console_formatter.py - A (80.81)
src\presentation\formatters\__init__.py - A (100.00)
src\presentation\mappers\map_generator.py - A (74.46)
src\presentation\mappers\__init__.py - A (100.00)
src\services\external_api_facade.py - A (83.92)
src\services\__init__.py - A (100.00)
src\services\api\geocoding_service.py - A (92.40)
src\services\api\routing_service.py - A (89.42)
src\services\api\__init__.py - A (100.00)
```

Modulo	Grado	MI Score	Stato
src/config.py	A	62.74	Passato
src/main.py	A	61.08	Passato

src/presentation/cli/argument_parser.py	A	100.00	Passato
src/presentation/formatters/console_formatter.py	A	80.81	Passato
src/presentation/mappers/map_generator.py	A	74.46	Passato

Scala di riferimento:

Grado	MI	Significato
A	20–100	Alta manutenibilità
B	10–19	Media — da monitorare
C	0–9	Bassa — richiede refactoring

6.3 Complessità Ciclomatica (radon cc src/ -s -a)

Classe / Metodo	Grado	CC	Stato
src.config.AppConfig.from_env_and_file	A	4	Passato
src.config.AppConfig	A	3	Passato
src.config.PathConfig	A	2	Passato
src.config.PathConfig.get_project_root	A	1	Passato
src.config.PathConfig.get_data_root	A	1	Passato
src.config.PathConfig.data_root	A	1	Passato
src.config.PathConfig.config_dir	A	1	Passato
src.config.PathConfig.input_dir	A	1	Passato
src.config.PathConfig.cache_dir	A	1	Passato
src.config.PathConfig.output_dir	A	1	Passato
src.config.PathConfig.report_dir	A	1	Passato
src.config.PathConfig.localita_file	A	1	Passato
src.config.PathConfig.quartieri_file	A	1	Passato
src.config.PathConfig.app_config_file	A	1	Passato
src.config.PathConfig.studenti_file	A	1	Passato
src.config.PathConfig.coordinate_cache_file	A	1	Passato
src.config.PathConfig.percorsi_cache_file	A	1	Passato
src.config.PathConfig.mappa_html_file	A	1	Passato

src.config.AppConfig.to_dict	A	1	Passato
src.config.AppConfig.print_summary	A	1	Passato
src.main.execute_optimize_mode	A	5	Passato
src.main.main	A	5	Passato
src.main.execute_populate_mode	A	3	Passato
src.main.initialize_layer1_data	A	1	Passato
src.main.initialize_layer2_services	A	1	Passato
src.main.initialize_layer3_business	A	1	Passato
src.main.initialize_layer4_presentation	A	1	Passato
src.presentation.cli.argument_parser.CLIArgumentParser	A	2	Passato
src.presentation.cli.argument_parser.ProgramConfig	A	1	Passato
src.presentation.cli.argument_parser.CLIArgumentParser.parse	A	1	Passato
src.presentation.formatters.console_formatter.ConsoleFormatter.format_flotta	A	4	Passato
src.presentation.formatters.console_formatter.ConsoleFormatter	A	3	Passato
src.presentation.formatters.console_formatter.ConsoleFormatter.__init__	A	1	Passato
src.presentation.formatters.console_formatter.ConsoleFormatter.print_flotta	A	1	Passato
src.presentation.mappers.map_generator.MapGenerator._add_equipaggio_to_map	B	7	Passato
src.presentation.mappers.map_generator.MapGenerator	A	5	Passato
src.presentation.mappers.map_generator.MapGenerator.generate	A	3	Passato
src.presentation.mappers.map_generator.MapGenerator.__init__	A	1	Passato

Complessità ciclomatica media complessiva: **A (1.89)**.

Scala di riferimento:

Grado	CC	Significato
A	1–5	Semplice, facile da testare
B	6–10	Moderata — accettabile
C	11–20	Alta — candidato a refactoring

D–F	>20	Critica — non accettabile

6.4 Qualità Codice (pylint src/)

Score: 8.59/10 (target: ≥ 8.5/10)

Tabella degli avvisi rilevanti:

Tipo	Codice	Messaggio	File	Priorità
Convention	C0301	Line too long (108/100)	src/config.py:216, :219	Bassa
Refactor	R0902	Too many instance attributes (11/7)	src/config.py:128 (AppConfig)	Media
Warning	W0718	Catching too general exception Exception	src/main.py:153, :224, :318,	Media

6.5 Riepilogo Metriche

Metrica	Target	Risultato	Stato
Statement Coverage	≥ 85%	**89%**	Passato
Branch Coverage	≥ 80%	**94.8%**	Passato
Maintainability Index	≥ 70 (A)	**75.82 (tutti A)**	Passato
Cyclomatic Complexity	≤ 10 (≤ B)	**Media A(1.89) – Max B(9)**	Passato
Pylint Score	≥ 8.5/10	**8.59/10**	Passato

7. Installazione ed Esecuzione

Questa sezione descrive i passi minimi per installare le dipendenze del progetto ed eseguire il programma nelle due modalità previste.

7.1 Prerequisiti

- **Python** 3.10 o superiore installato nel sistema.
- Accesso a Internet per la modalità popola (chiamate a servizi esterni).
- Chiave API valida per OpenRouteService, esposta come variabile d'ambiente ORS_API_KEY.

7.2 Installazione dipendenze

Si consiglia di lavorare in un **virtual environment** dedicato (es. python -m venv .venv e poi attivazione), quindi installare il progetto in modalità *editable*:

```
pip install -e .
```

Questo comando:

- installa le dipendenze dichiarate dal progetto;
- rende il package importabile come modulo Python durante lo sviluppo (senza reinstallare a ogni modifica).

7.3 Configurazione variabili d'ambiente

Impostare la chiave API ORS_API_KEY prima di eseguire il programma, ad esempio:

```
# Linux / macOS
export ORS_API_KEY="la-tua-chiave-api"

# Windows (PowerShell)
$Env:ORS_API_KEY="la-tua-chiave-api"
```

In alternativa si può usare un file .env gestito da python-dotenv, purché contenga:

```
ORS_API_KEY=la-tua-chiave-api
```

7.4 Esecuzione del programma

Dal root del progetto:

- **Modalità POPOLA** (scarica dati da API e popola la cache; richiede rete):

```
python src/main.py popola
```

- **Modalità OTTIMIZZA** (usa la cache per calcolare gli equipaggi e generare la mappa HTML; può lavorare offline se la cache è già pronta):

```
python src/main.py ottimizza
```

In entrambe le modalità:

- main() carica la configurazione tramite AppConfig.from_env_and_file();
- viene mostrato un riepilogo dei parametri principali (capienza auto, numero cluster, ecc.);
- il programma prosegue con l'inizializzazione dei quattro layer e il routing della modalità richiesta.